

DataFrame-Joins: Inner, Left, Broadcast, Cross und Union

Section 3 · Data Transformation and Modelling · Quelle: PySpark-/Databricks-Referenzdokumentation

Kaum eine Transformation ist im Data-Engineering-Alltag so häufig wie das Zusammenführen zweier Tabellen. Der Exam Guide verlangt konkret die Beherrschung von Inner Join, Left Join, Broadcast Join, Joins über mehrere Schlüssel, Cross Join sowie `union / union all`. Dieses Kapitel zeigt alle genannten Varianten anhand von zwei einfachen Beispieltabellen – `customers` (Kunden) und `orders` (Bestellungen) – jeweils in PySpark- und SQL-Syntax.

Als Ausgangsdaten dienen zwei kleine, frei erfundene DataFrames:

```
customers = spark.createDataFrame([
    (1, "Anna", "Berlin"),
    (2, "Ben", "München"),
    (3, "Clara", "Hamburg"),
], ["customer_id", "name", "city"])

orders = spark.createDataFrame([
    (100, 1, 49.90),
    (101, 1, 12.50),
    (102, 2, 99.00),
    (103, 4, 20.00), # customer_id 4 existiert nicht in customers
], ["order_id", "customer_id", "amount"])

customers.createOrReplaceTempView("customers")
orders.createOrReplaceTempView("orders")
```

1. Inner Join

Der **Inner Join** ist der Standardfall: Er liefert ausschließlich Zeilen, für die in *beiden* DataFrames ein passender Schlüssel existiert. Kunde „Clara“ (ohne Bestellung) und Bestellung 103 (mit unbekanntem `customer_id` 4) fallen dabei aus dem Ergebnis heraus.

```
result = customers.join(orders, on="customer_id", how="inner")
# gleichwertig: how="inner" ist der Standardwert und kann weggelassen werden
result = customers.join(orders, on="customer_id")
```

```
SELECT c.customer_id, c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

2. Left Join (Left Outer Join)

Ein **Left Join** (auch `left_outer`) behält alle Zeilen der linken Tabelle bei – unabhängig davon, ob ein passender Partner in der rechten Tabelle existiert. Fehlt ein Treffer, werden die Spalten der rechten Tabelle mit `NULL` aufgefüllt. Im Beispiel bleibt Clara im Ergebnis erhalten, ihre Bestellspalten sind jedoch `NULL`.

```
result = customers.join(orders, on="customer_id", how="left")
# alternative Schreibweisen, alle gleichwertig:
customers.join(orders, on="customer_id", how="left_outer")
```

```
SELECT c.customer_id, c.name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Analog existieren `how="right"` (Right Outer Join, behält alle Zeilen der rechten Tabelle) und `how="outer"` bzw. `"full"` (Full Outer Join, behält Zeilen aus beiden Tabellen und füllt jeweils fehlende Seite mit `NULL` auf).

3. Broadcast Join

Ein **Broadcast Join** ist keine eigene `how`-Option, sondern ein *Optimierungshinweis*: Ist eine der beiden Tabellen klein genug, um komplett in den Arbeitsspeicher jedes Executors zu passen (typischerweise eine kleine Dimensionstabelle wie `customers`), kann sie vollständig an alle Executor verteilt werden. Das erspart den teuren Shuffle der großen Tabelle über das Netzwerk. Spark entscheidet automatisch anhand des Parameters `spark.sql.autoBroadcastJoinThreshold` (siehe Kapitel 14 dieser Section), ob gebroadcastet wird – mit dem expliziten Hint `broadcast()` lässt sich dieses Verhalten aber auch gezielt erzwingen.

```
from pyspark.sql.functions import broadcast

result = orders.join(broadcast(customers), on="customer_id", how="inner")
```

```
SELECT /*+ BROADCAST(c) */ o.order_id, o.amount, c.name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Der Broadcast-Hint eignet sich vor allem dann, wenn Spark die tatsächliche Größe einer Tabelle nicht korrekt einschätzen kann (z. B. nach mehreren Transformationsschritten) und deshalb von sich aus keinen Broadcast Join wählen würde, obwohl die Tabelle klein genug wäre.

4. Joins über mehrere Schlüssel

Sollen zwei Tabellen über *mehrere* Spalten gleichzeitig verglichen werden (z. B. weil ein einzelner Schlüssel nicht eindeutig ist), gibt es in PySpark zwei gängige Varianten: eine Liste gemeinsamer Spaltennamen oder eine zusammengesetzte Bedingung mit `&`.

```
# Variante A: Liste gemeinsamer Spaltennamen (identisch benannt in beiden DataFrames)
result = df1.join(df2, on=["customer_id", "order_year"], how="inner")

# Variante B: zusammengesetzte Bedingung, auch bei unterschiedlichen Spaltennamen möglich
result = df1.join(
    df2,
    on=(df1.customer_id == df2.cust_id) & (df1.order_year == df2.year),
    how="inner",
)
```

```
SELECT *
FROM df1
INNER JOIN df2
  ON df1.customer_id = df2.cust_id
  AND df1.order_year = df2.year;
```

Wichtig bei Variante B: Da die Join-Bedingung hier über einen Spaltenausdruck statt über `on="..."` mit gemeinsamem Namen erfolgt, tauchen beide Schlüsselspalten (`customer_id` und `cust_id`) im Ergebnis auf – im Gegensatz zu Variante A, bei der die gemeinsame Spalte nur einmal erscheint.

5. Cross Join

Ein **Cross Join** (kartesisches Produkt) verknüpft jede Zeile der linken Tabelle mit jeder Zeile der rechten Tabelle – ganz ohne Join-Bedingung. Die Ergebnisgröße entspricht dem Produkt beider Zeilenzahlen und kann dadurch sehr schnell sehr groß werden. Typischer Anwendungsfall sind Kombinationstabellen, etwa alle möglichen Kombinationen aus Produktfarben und -größen.

```
colors = spark.createDataFrame([("rot",), ("blau",)], ["farbe"])
sizes  = spark.createDataFrame([("S",), ("M",), ("L",)], ["groesse"])

kombinationen = colors.crossJoin(sizes)    # Ergebnis: 2 x 3 = 6 Zeilen
```

```
SELECT farbe, groesse
FROM colors
CROSS JOIN sizes;
```

Cross Joins sollten mit Bedacht eingesetzt werden: Bei zwei Tabellen mit jeweils einer Million Zeilen entstünde ein Ergebnis mit einer Billion Zeilen. Spark verweigert einen impliziten Cross Join standardmäßig sogar (Fehlermeldung `CROSS_JOIN_UNSUPPORTED`), wenn `join()` ohne Bedingung aufgerufen wird – die explizite Methode `crossJoin()` bzw. die SQL-Klausel `CROSS JOIN` muss bewusst gewählt werden.

6. Union, Union All und UnionByName

Während Joins Tabellen *nebeneinander* (spaltenweise) zusammenführen, hängen `union`-Operationen zwei DataFrames *untereinander* (zeilenweise) an – Voraussetzung ist dieselbe Anzahl an Spalten mit kompatiblen Datentypen.

```

orders_de = spark.createDataFrame([(200, 1, 30.0)], ["order_id", "customer_id", "amount"])
orders_at = spark.createDataFrame([(300, 2, 45.0)], ["order_id", "customer_id", "amount"])

# union() und unionAll() sind in PySpark identisch (kein automatisches DISTINCT!)
alle_bestellungen = orders_de.union(orders_at)
alle_bestellungen = orders_de.unionAll(orders_at)  # Alias, gleiches Verhalten

# unionByName(): matcht Spalten anhand ihres Namens statt ihrer Position
orders_ch = spark.createDataFrame([(2, 400, 60.0)], ["customer_id", "order_id", "amount"])
alle_bestellungen = orders_de.unionByName(orders_ch)

# unionByName mit allowMissingColumns=True erlaubt sogar unterschiedliche Spaltenmengen
# fehlende Spalten werden in den betroffenen Zeilen automatisch mit NULL aufgefüllt
orders_uk = spark.createDataFrame([(500, 3, 15.0, "GBP")], ["order_id", "customer_id",
"amount", "currency"])
alle_bestellungen = orders_de.unionByName(orders_uk, allowMissingColumns=True)

```

```

SELECT * FROM orders_de
UNION ALL
SELECT * FROM orders_at;

-- SQL UNION (ohne ALL) entfernt zusätzlich Duplikate, entspricht also union().distinct()
SELECT * FROM orders_de
UNION
SELECT * FROM orders_at;

```

Wichtiger Unterschied zwischen PySpark und SQL: In der DataFrame-API sind `union()` und `unionAll()` identisch und entfernen *keine* Duplikate. In reinem SQL hingegen entfernt `UNION` (ohne `ALL`) Duplikate, während `UNION ALL` alle Zeilen unverändert übernimmt – hier lohnt sich also besondere Aufmerksamkeit, da der Name `union()` in PySpark auf den ersten Blick das SQL-`UNION`-Verhalten vermuten lässt, tatsächlich aber `UNION ALL` entspricht. Ein anschließendes `.distinct()` stellt bei Bedarf das SQL-`UNION`-Verhalten in PySpark her. `union()` und `unionAll()` matchen Spalten zudem rein **positionsbasiert** – unterscheidet sich die Spaltenreihenfolge zwischen beiden DataFrames, landen Werte unbemerkt in der falschen Spalte. `unionByName()` vermeidet dieses Risiko, indem es Spalten anhand ihres Namens zuordnet.

7. Welcher Join-Typ wann?

Join-Typ	Wann sinnvoll
Inner Join	Nur eindeutig zuordenbare Datensätze aus beiden Tabellen werden benötigt (Standardfall).

Left Join	Alle Datensätze der „Haupttabelle“ sollen erhalten bleiben, auch ohne Treffer in der Zweittabelle (z. B. alle Kunden, auch ohne Bestellung).
Broadcast Join	Eine der beiden Tabellen ist klein genug für den Arbeitsspeicher jedes Executors (Dimensionstabelle) – spart teuren Shuffle.
Mehrfachschlüssel-Join	Ein einzelner Schlüssel reicht zur eindeutigen Zuordnung nicht aus (z. B. Kombination aus ID und Jahr).
Cross Join	Bewusst alle Kombinationen zweier (meist kleiner) Mengen erzeugt werden sollen.
Union / Union All	Zeilen mehrerer strukturgleicher Tabellen sollen untereinandergehängt werden, z. B. regionale Teiltabellen zu einer Gesamttabelle.

Ergänzt aus der offiziellen PySpark-Dokumentation: `DataFrame.crossJoin()` erzeugt explizit das kartesische Produkt zweier DataFrames ohne Join-Bedingung. `functions.broadcast()` markiert einen DataFrame als klein genug, um an alle Worker-Knoten verteilt zu werden, und ist damit der explizite Gegenpart zum automatischen Broadcast-Verhalten über `spark.sql.autoBroadcastJoinThreshold`. `DataFrame.unionByName()` unterstützt seit Spark 3.1 zusätzlich den Parameter `allowMissingColumns`, der fehlende Spalten in einem der beiden DataFrames automatisch mit `NULL` auffüllt, statt einen Fehler zu werfen.
Quelle: [pyspark.sql.DataFrame.crossJoin – Apache Spark Dokumentation](#)